Disciplina:
Dynamic Programming

Professor:
André Luís Ferreira Marques

Integrantes:

Gabriel Alexandre Fukushima Sakura	RM: 99522
Henrique de Oliveira Gomes	RM: 563467
Henrique Kolomyes Silveira	RM: 563467
Lucas Henrique Viana Estevam Sena	RM: 566246
Matheus Santos de Oliveira	RM: 561982

Sistema de Priorização de Pedidos Logísticos

1. Introdução

Em sistemas logísticos modernos, a eficiência na organização e processamento de pedidos é essencial para garantir prazos de entrega e otimizar recursos. Nesse contexto, a priorização de pedidos torna-se um fator estratégico, pois permite direcionar esforços para demandas mais críticas.

Este trabalho apresenta o desenvolvimento de um sistema em Python capaz de simular um centro de distribuição, organizando pedidos com base em critérios relevantes como urgência, tempo estimado de entrega, status de pagamento e quantidade de itens.

A proposta integra conceitos fundamentais de programação, como estruturas de dados, algoritmos de ordenação e análise de complexidade, aplicando-os em um cenário prático inspirado em problemas reais de logística.

2. Objetivo

O objetivo deste projeto é desenvolver uma solução computacional que permita:

- Ler e processar dados logísticos a partir de um arquivo CSV
- Aplicar critérios de priorização em pedidos
- Organizar o processamento em uma fila eficiente
- Utilizar diferentes estruturas de dados de forma estratégica
- Avaliar a eficiência da solução por meio de análise de complexidade

3. Modelagem da Solução

A solução foi estruturada como um fluxo de processamento de dados, no qual cada etapa possui uma responsabilidade específica. Esse modelo permite transformar dados brutos em uma estrutura organizada para tomada de decisão.

O fluxo do sistema pode ser representado da seguinte forma:

CSV -> DataFrame -> Lista -> Tuplas -> Dicionário -> Priorização -> Ordenação -> Fila -> Processamento

Inicialmente, os dados são carregados a partir de um arquivo CSV e convertidos para um DataFrame. Em seguida, passam por transformações sucessivas até serem organizados em uma fila de processamento baseada em prioridade.

Essa abordagem modular facilita a manutenção do código e permite compreender claramente cada etapa do processamento.

4. Estruturas de Dados Utilizadas

O desenvolvimento do sistema envolveu a utilização de múltiplas estruturas de dados, cada uma escolhida com base em suas características e vantagens.

O DataFrame foi utilizado na etapa inicial devido à sua eficiência na leitura e manipulação de dados tabulares. Em seguida, os dados foram convertidos em listas para facilitar a iteração e transformação.

As tuplas foram adotadas como forma de representar os pedidos de maneira imutável, garantindo maior segurança contra alterações acidentais durante o processamento.

```python
# Função que transforma o DataFrame em uma lista
def df_para_lista(df):
    return df.values.tolist()


# Função que transforma cada item da lista em tupla (estrutura imutável)
def lista_para_tuplas(lista):
    return [tuple(item) for item in lista]
```

Para organização e acesso rápido, foi utilizado um dicionário, permitindo recuperar pedidos pelo identificador em tempo constante (O(1)).

```python
# Função que organiza os pedidos em um dicionário para facilitar a busca por ID
def criar_dicionario(lista_pedidos):
    dicionario = {}

    for pedido in lista_pedidos:
        id_pedido = pedido[0]

        dicionario[id_pedido] = {
            "cidade_destino": pedido[1],
            "produto": pedido[2],
            "categoria": pedido[3],
            "quantidade": pedido[4],
            "valor_unitario": pedido[5],
            "urgencia": pedido[6],
            "tempo_estimado_horas": pedido[7],
            "modal": pedido[8],
            "status_pagamento": pedido[9]
        }

    return dicionario
```

```
Dicionário de pedidos:
1: {'cidade_destino': 'SP', 'produto': 'Arroz', 'categoria': 'Alimento', 'quantidade': 100, 'valor_unitario': 5,
 'urgencia': 'alta', 'tempo_estimado_horas': 10, 'modal': 'rodoviario', 'status_pagamento': 'ok'}
2: {'cidade_destino': 'RJ', 'produto': 'Feijao', 'categoria': 'Alimento', 'quantidade': 200, 'valor_unitario': 6
, 'urgencia': 'media', 'tempo_estimado_horas': 20, 'modal': 'ferroviario', 'status_pagamento': 'ok'}
3: {'cidade_destino': 'MG', 'produto': 'Milho', 'categoria': 'Alimento', 'quantidade': 150, 'valor_unitario': 4,
 'urgencia': 'baixa', 'tempo_estimado_horas': 15, 'modal': 'rodoviario', 'status_pagamento': 'pendente'}
4: {'cidade_destino': 'BA', 'produto': 'Soja', 'categoria': 'Alimento', 'quantidade': 300, 'valor_unitario': 7,
'urgencia': 'alta', 'tempo_estimado_horas': 12, 'modal': 'ferroviario', 'status_pagamento': 'ok'}
5: {'cidade_destino': 'PR', 'produto': 'Cafe', 'categoria': 'Alimento', 'quantidade': 250, 'valor_unitario': 10,
 'urgencia': 'media', 'tempo_estimado_horas': 18, 'modal': 'rodoviario', 'status_pagamento': 'ok'}
```

Por fim, a estrutura deque foi empregada para simular a fila de processamento, pois oferece operações eficientes de inserção e remoção, ambas com complexidade O(1), sendo mais adequada do que listas para esse tipo de aplicação.

```python
# Função que cria uma fila priorizada usando deque
def criar_fila_priorizada(lista_ordenada):
    fila = deque([pedido for pedido, _ in lista_ordenada])
    return fila
```

5. Algoritmo de Priorização

A priorização dos pedidos é o núcleo do sistema. Para isso, foi desenvolvido um algoritmo baseado em uma abordagem heurística, no qual diferentes critérios são convertidos em valores numéricos e combinados em um único score.

Os principais fatores considerados foram:

- nível de urgência do pedido
- tempo estimado de entrega
- status do pagamento
- quantidade de itens

Cada um desses critérios contribui para o cálculo final da prioridade, permitindo transformar características qualitativas em um valor quantitativo comparável.

Essa abordagem possibilita ordenar os pedidos de forma objetiva, mesmo quando múltiplos fatores estão envolvidos. Embora simplificada, a heurística representa uma aproximação eficiente de decisões logísticas reais.

```python
# Função que calcula a prioridade de um pedido com base em múltiplos critérios
def calcular_prioridade(pedido):
    # Convertendo urgência para valor numérico
    urgencia = {"alta": 3, "media": 2, "baixa": 1}[pedido[6]]

    # Definindo peso baseado no tempo estimado
    demora = 3 if pedido[7] > 15 else 2 if pedido[7] > 10 else 1

    # Verificando status de pagamento
    importancia = 2 if pedido[9] == "ok" else 1

    # Considerando quantidade como fator adicional
    quantidade = pedido[4] / 10

    # Retornando score final do pedido
    return urgencia * 2 + importancia * 2 + demora + quantidade
```

6. Ordenação e Eficiência

Após o cálculo do score de prioridade, os pedidos são ordenados utilizando a função sorted() do Python, que implementa o algoritmo Timsort.

Esse algoritmo apresenta complexidade O(n log n), sendo eficiente para grandes volumes de dados e amplamente utilizado em aplicações reais.

A ordenação é essencial para transformar a análise de prioridade em uma estrutura prática de execução, garantindo que os pedidos mais relevantes sejam processados primeiro.

```
Top 5 pedidos priorizados:
Score: 42.0 | Pedido: (4, 'BA', 'Soja', 'Alimento', 300, 7, 'alta', 12, 'ferroviario', 'ok')
Score: 36.0 | Pedido: (5, 'PR', 'Cafe', 'Alimento', 250, 10, 'media', 18, 'rodoviario', 'ok')
Score: 31.0 | Pedido: (2, 'RJ', 'Feijao', 'Alimento', 200, 6, 'media', 20, 'ferroviario', 'ok')
Score: 21.0 | Pedido: (1, 'SP', 'Arroz', 'Alimento', 100, 5, 'alta', 10, 'rodoviario', 'ok')
Score: 21.0 | Pedido: (3, 'MG', 'Milho', 'Alimento', 150, 4, 'baixa', 15, 'rodoviario', 'pendente')
```

7. Simulação da Fila de Processamento

Com os pedidos já ordenados, o sistema os insere em uma fila utilizando a estrutura deque. O processamento segue o modelo FIFO (First In, First Out), no qual o primeiro elemento da fila é o primeiro a ser removido.

Essa abordagem simula o funcionamento de um sistema logístico real, onde pedidos são processados sequencialmente, respeitando a ordem definida previamente.

A escolha do deque garante eficiência, pois a remoção de elementos do início da fila ocorre em tempo constante, diferentemente de listas tradicionais.

```
Processamento de pedidos:
Processando pedido: (4, 'BA', 'Soja', 'Alimento', 300, 7, 'alta', 12, 'ferroviario', 'ok')
Processando pedido: (5, 'PR', 'Cafe', 'Alimento', 250, 10, 'media', 18, 'rodoviario', 'ok')
Processando pedido: (2, 'RJ', 'Feijao', 'Alimento', 200, 6, 'media', 20, 'ferroviario', 'ok')
Processando pedido: (1, 'SP', 'Arroz', 'Alimento', 100, 5, 'alta', 10, 'rodoviario', 'ok')
Processando pedido: (3, 'MG', 'Milho', 'Alimento', 150, 4, 'baixa', 15, 'rodoviario', 'pendente')
```

8. Tratamento de Erros e Uso de Recursão

O sistema inclui tratamento de erros na leitura do arquivo CSV, utilizando exceções para evitar falhas durante a execução.

Em caso de caminho inválido, é aplicada uma abordagem recursiva, na qual a função solicita novamente o caminho ao usuário até que um valor válido seja fornecido.

Essa estratégia torna o sistema mais robusto e melhora a experiência do usuário, evitando encerramentos inesperados.

```python
# Função responsável por carregar os dados do arquivo CSV
def carregar_dados(caminho):
    try:
        df = pd.read_csv(caminho)
        return df
    except FileNotFoundError:
        print("Caminho inválido!")
        novo_caminho = input("Digite o caminho correto do csv: ")
        return carregar_dados(novo_caminho)  # Chamada recursiva
```

9. Visualização de Dados

Para complementar a análise, foram gerados gráficos utilizando a biblioteca matplotlib. Entre as visualizações produzidas estão a distribuição de urgência dos pedidos e o ranking dos pedidos mais prioritários.

Essas representações visuais permitem uma melhor compreensão dos dados, facilitando a identificação de padrões e auxiliando na tomada de decisão.

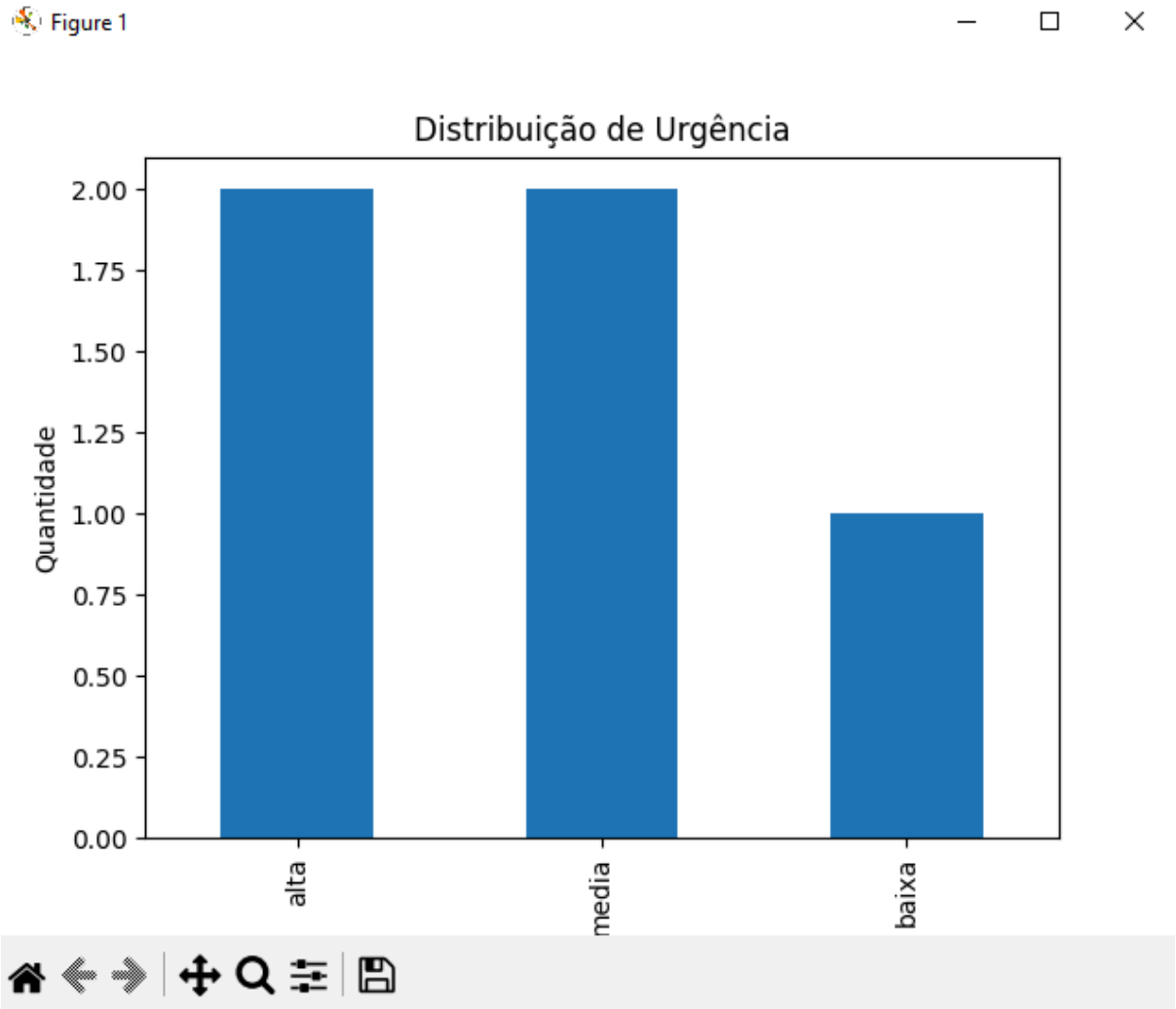
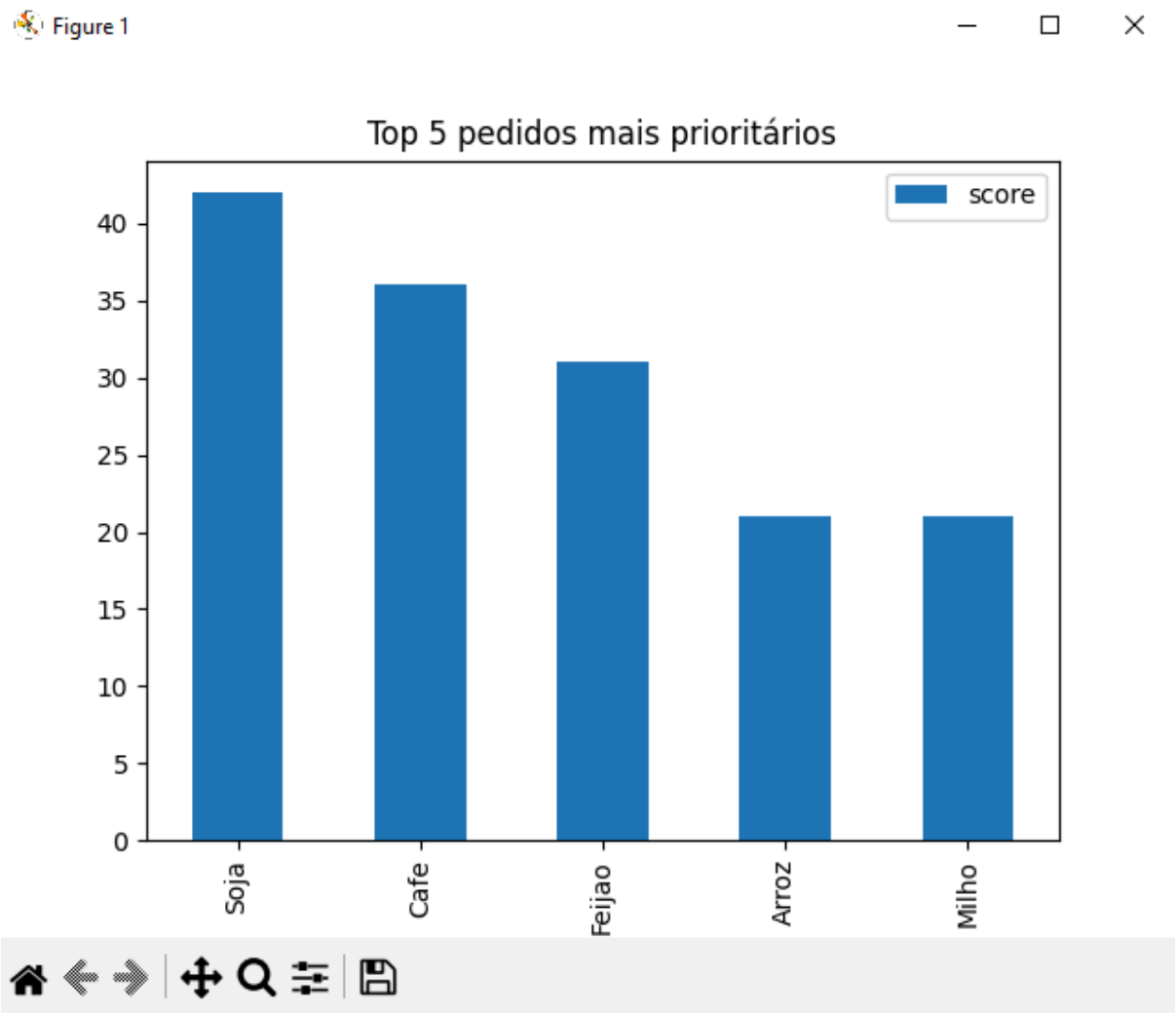


Figure 1
Distribuição de Urgência
Quantidade
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
alta
media
baixa



10. Análise de Complexidade

A solução foi projetada com foco em eficiência computacional. As principais etapas apresentam as seguintes complexidades:

- Leitura e transformação de dados: O(n)
- Cálculo de prioridade: O(n)
- Ordenação dos pedidos: O(n log n)
- Operações na fila (deque): O(1)

A etapa mais custosa é a ordenação, enquanto as demais mantêm desempenho linear ou constante, garantindo boa escalabilidade do sistema.

11. Conclusão

O desenvolvimento deste projeto permitiu aplicar conceitos fundamentais de estruturas de dados e algoritmos em um contexto prático e relevante.

A utilização de diferentes estruturas mostrou-se essencial para equilibrar desempenho, organização e segurança dos dados. Além disso, o algoritmo de priorização demonstrou como critérios do mundo real podem ser traduzidos em decisões computacionais.

Por fim, o sistema evidencia a importância da eficiência na construção de soluções escaláveis, especialmente em cenários que envolvem grande volume de dados, como a logística.

12. Melhorias Futuras

Como possíveis evoluções do sistema, destacam-se:

- implementação de fila de prioridade (heap)
- ajuste dinâmico dos pesos de priorização
- integração com banco de dados
- desenvolvimento de interface gráfica
- criação de uma API para integração com outros sistemas